

Eye: Program Visualizer for CS2

Aman Bansal
aman0456b@gmail.com
Indian Institute of Technology Bombay

Preey Shah
preeyshah@gmail.com
Indian Institute of Technology Bombay

Sahil Shah
sahilshah00199@gmail.com
Indian Institute of Technology Bombay

ABSTRACT

In recent years, programming has witnessed a shift towards using standard libraries as a black box. However, there has not been a synchronous development of tools that can help demonstrate the working of such libraries in general programs, which poses an impediment to improved learning outcomes and makes debugging exasperating. We introduce *Eye*, an interactive pedagogical tool that visualizes a program’s execution as it runs. It demonstrates properties and usage of data structures in a general environment, thereby helping in learning, logical debugging, and code comprehension. *Eye* provides a comprehensive overview at each stage during run time including the execution stack and the state of data structures. The modular implementation allows for extension to other languages and modification of the graphics as desired.

Eye opens up a gateway for CS2 students to more easily understand myriads of programs that are available on online programming websites, lowering the barrier towards self-learning of coding. It expands the scope of visualizing data structures from standard algorithms to general cases, benefiting both teachers as well as programmers who face issues in debugging. Line by line interpreting allows *Eye* to describe the execution and not only the current state. We also conduct experiments to evaluate the efficacy of *Eye* for debugging and comprehending a new piece of code. Our findings show that it becomes faster and less frustrating to debug certain problems using this tool, and also makes understanding new code a much more pleasant experience.

1 PROBLEM AND MOTIVATION

With the increasing popularity of Computer Science (CS) and the advent of CoViD-19, instructors now need to both effectively teach over video conference and empower students to continue learning on their own. This requires tools that can facilitate self-learning and allow students to use existing programming resources (such as online programming websites). This problem is more pronounced for novice programmers, who are categorized by: (i) Having sufficient introductory programming knowledge (‘CS1’ curriculum equivalent), (ii) Currently learning data structures and algorithms (‘CS2’ curriculum equivalent), and (iii) Practicing programming problems to hone programming skills. We enumerate some specific challenges faced by such *beginners*:

- (1) *Learning*: The primary challenge is learning the working of well-known data structures and algorithms.
- (2) *Logical Debugging*: The second challenge is debugging errors that cause the program to give wrong results without throwing any error. This process is difficult and frustrating [4, 6, 7]. Furthermore, an incorrect understanding or usage of standard libraries can accentuate this problem.
- (3) *Code comprehension*: The third challenge is understanding a given code. Often, beginners can’t solve a problem and the most readily, sometimes the only, available option is to find

a working code and understand it. This is markedly more pronounced for problems on online programming websites.

To mitigate these problems, we introduce a tool *Eye* which is an interactive program visualizer. For any such tool to be effective and widespread, the following properties must be satisfied:

- P1 **Completeness**: It should support CS2-equivalent courses by visualizing data structures classes (such as C++ STL). This is required for covering the CS curriculum of beginners.
- P2 **Flexibility**: It should provide flexibility to change the display (beyond trivial CSS based changes like colors and fonts) so that the instructor can modify it without much trouble. Examples include changing the layout or granularity of execution steps.
- P3 **Awareness**: It should allow the addition of ‘program-aware’ features, including explanatory text and variable scoping, which would improve understanding of programs.
- P4 **Accessibility**: It should run and display the visual elements in a web browser to ensure that every user can use it from anywhere without any hassle of installation or compatibility.
- P5 **Modularity**: It should support multiple languages or be modular enough to support new languages with minimal back-end changes to cater to the needs of different instructors without changes to the remaining implementation.
- P6 **Interactivity**: It should be interactive, allowing the beginners to go back and forth as per their convenience. This is required for better learning [12, 14].

2 RELATED WORK

The concept of using visualizations is not new. Studies have shown that the amount of time students spend on interactive visualization tools correlates with their performance [2, 5]. [10] and [11] give an overview of many visualization tools; however, when scrutinized closely, these do not satisfy all the properties listed above. We give a comparison with some of the prominent program visualization tools in Table 1.

Table 1: Comparison with Existing Visualization Tools

Tools	P1	P2	P3	P4	P5	P6
Jeliot 3 [1]	✗	✗	✗	✗	✗	✓
jGRASP [3]	✓	✗	✗	✗	✗	✗
Jsvee [9]	✗	✓	✓	✓	✓	✓
Python Tutor [2]	✗	✗	✗	✓	✓	✓
UUhistle [13]	✗	✗	✓	✗	✗	✓
VILLE [8]	✗	✗	✓	✗	✓	✓
<i>Eye</i>	✓	✓	✓	✓	✓	✓

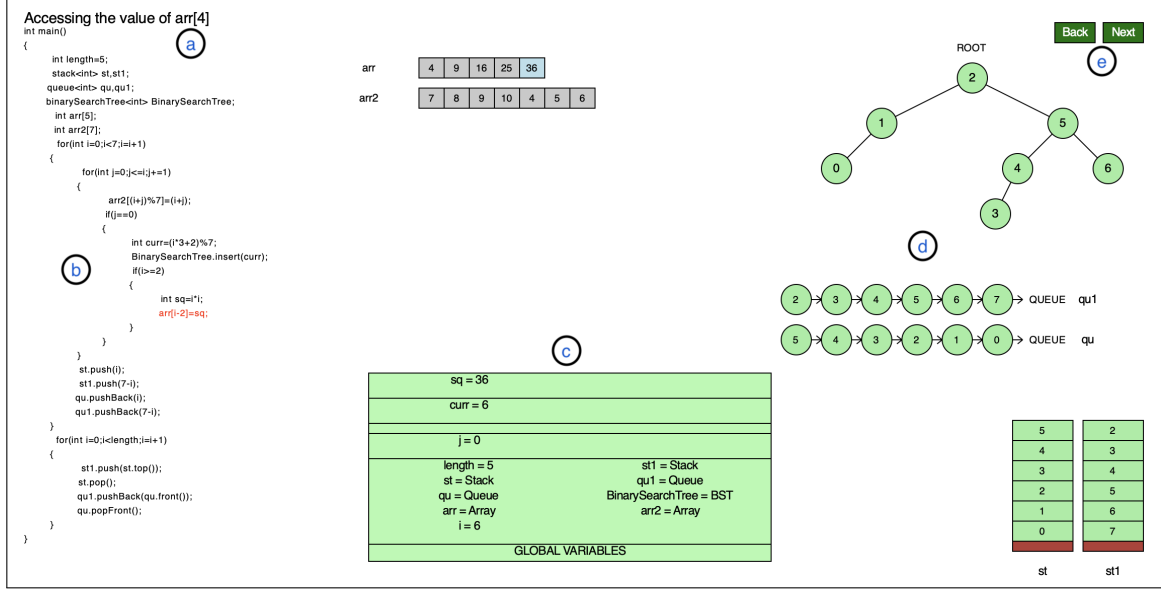


Figure 1: Main Screen of *Eye*. (a) Explanation of code being executed. (b) Code snippet being executed in red. Data structures have been named intuitively for better understanding. (c) Variables in the execution stack, with a different block for each scope. (d) Visualization of data structures like queue, stack, binary search tree, and array. (e) Back and forward buttons.

3 DESIGN OVERVIEW

In this section, we describe the functionality provided by our tool, whose primary window is shown in Figure 1 with some fundamental elements on the canvas.

- An execution stack that shows all the variables and their current values. We divide the stack into different sections for representing different scopes to elucidate the concept of scoping and expediting the detection of bugs due to *variable shadowing*, a feature that was missing from other tools.
- A new execution stack as soon as a new function starts executing and a different color for the currently active frame.
- Besides displaying the source code with the current line highlighted, we provide an explanatory line summarizing the operation being executed.
- Multiple data structures such as vector (array), map (binary search tree), stack, queue, deque, and unordered_map (hash table) are supported. This lets beginners grasp the working of these data structures and verify their state while debugging.
- Every access to these data structures is highlighted (including arrays). This speeds up the debugging process as students can skip the code or explanation and directly verify if all accesses (and assignments) are occurring as expected. Indexing errors become noticeable due to this feature.
- We visually explain what happens internally in each data structure on a function call. For example, when an element is inserted or deleted in a binary search tree.

4 EXPERIMENTAL RESULTS

We design experiments to determine whether (a) using *Eye* for debugging data structures based programs accelerates the process and

Table 2: Summary of the results of both experiments

	Exp1 (Avg % time)		Exp2 (Avg % marks)
	Prob1	Prob2	
Without <i>Eye</i>	58.3	51.4	59.2
With <i>Eye</i>	48.6	41.7	60.6
<i>p</i> -value	0.0578		0.446

(b) using *Eye* to understand new code improves code comprehension. To determine (a), 20 senior computer science undergraduates were split into 2 groups and asked to debug 2 programs, one based on *stack* and one on *queue*. Group 1 debugged the first using *Eye* and the second without, whereas Group 2 did the opposite. To account for debugging skill bias, we calculate the percentage of total debug time the subjects spent on each problem. We consider these percentages to be random variables and test against the null hypothesis that the variables corresponding to the same question have the same mean for the two groups. The average values for this measure is shown in Table 2. The low *p*-value shows that *Eye* improves debug time.

For (b), students were once again independently divided into 2 groups and were given a problem statement with a correct solution implementation which they could read and execute for a fixed amount of time, post which they were given a time bound quiz to test their understanding of the algorithm. Though the average percentage score for the group using *Eye* was better (Table 2), the difference was not statistically significant on testing against the null hypothesis of the scores being drawn from the same distribution for both groups. However, based on anonymous feedback collected, students said that *Eye* would help reduce frustration, debugging time, and aid understanding once they become familiar with it.

REFERENCES

- [1] Mordechai Ben-Ari, Roman Bednarik, Ronit Levy, Gil Ebel, Andrés Moreno, Niko Myller, and Erkki Sutinen. 2011. A decade of research and development on program animation: The Jeliot experience. *J. Vis. Lang. Comput.* 22 (10 2011), 375–384. <https://doi.org/10.1016/j.jvlc.2011.04.004>
- [2] Philip J. Guo. 2013. Online Python Tutor: Embeddable Web-Based Program Visualization for Cs Education. In *Proceeding of the 44th ACM Technical Symposium on Computer Science Education* (Denver, Colorado, USA) (SIGCSE '13). Association for Computing Machinery, New York, NY, USA, 579–584. <https://doi.org/10.1145/2445196.2445368>
- [3] T. Dean Hendrix, James H. Cross, and Larry A. Barowski. 2004. An Extensible Framework for Providing Dynamic Data Structure Visualizations in a Lightweight IDE. *SIGCSE Bull.* 36, 1 (March 2004), 387–391. <https://doi.org/10.1145/1028174.971433>
- [4] Essi Lahtinen, Kirsti Ala-Mutka, and Hannu-Matti Järvinen. 2005. A Study of the Difficulties of Novice Programmers. In *Proceedings of the 10th Annual SIGCSE Conference on Innovation and Technology in Computer Science Education* (Caparica, Portugal) (ITiCSE '05). Association for Computing Machinery, New York, NY, USA, 14–18. <https://doi.org/10.1145/1067445.1067453>
- [5] Ronit Ben-Bassat Levy, Mordechai Ben-Ari, and Pekka A. Uronen. 2003. The Jeliot 2000 Program Animation System. *Comput. Educ.* 40, 1 (Jan. 2003), 1–15. [https://doi.org/10.1016/S0360-1315\(02\)00076-3](https://doi.org/10.1016/S0360-1315(02)00076-3)
- [6] D. N. Perkins, Chris Hancock, Renee Hobbs, Fay Martin, and Rebecca Simmons. 1986. Conditions of Learning in Novice Programmers. *Journal of Educational Computing Research* 2, 1 (1986), 37–55. <https://doi.org/10.2190/GUJT-JCBJ-Q6QU-Q9PL> arXiv:<https://doi.org/10.2190/GUJT-JCBJ-Q6QU-Q9PL>
- [7] Michael Perscheid, Benjamin Siegmund, Marcel Taeumel, and Robert Hirschfeld. 2017. Studying the Advancement in Debugging Practice of Professional Software Developers. *Software Quality Journal* 25, 1 (March 2017), 83–110. <https://doi.org/10.1007/s11219-015-9294-2>
- [8] Teemu Rajala, Mikko-Jussi Laakso, Erkki Kaila, and Tapio Salakoski. 2007. VILLE: A Language-Independent Program Visualization Tool. In *Proceedings of the Seventh Baltic Sea Conference on Computing Education Research - Volume 88* (Koli National Park, Finland) (Koli Calling '07). Australian Computer Society, Inc., AUS, 151–159.
- [9] Teemu Sirkia. 2016. Jsvee & Kelmu: Creating and Tailoring Program Animations for Computing Education. In *2016 IEEE Working Conference on Software Visualization (VISSOFT)* (Raleigh, NC, USA). IEEE, New York, NY, USA, 36–45. <https://doi.org/10.1109/VISSOFT.2016.24>
- [10] Juha Sorva. 2012. *Visual program simulation in introductory programming education; Visuaalinen ohjelmatisimulaatio ohjelmoinnin alkeisopetuksessa*. G4 Monografiaväitöskirja. Aalto University. <http://urn.fi/URN:ISBN:978-952-60-4626-6>
- [11] Juha Sorva, Ville Karavirta, and Lauri Malmi. 2013. A Review of Generic Program Visualization Systems for Introductory Programming Education. *ACM Trans. Comput. Educ.* 13, 4, Article 15 (Nov. 2013), 64 pages. <https://doi.org/10.1145/2490822>
- [12] Juha Sorva, Jan Lönnberg, and Lauri Malmi. 2013. Students' ways of experiencing visual program simulation. *Computer Science Education* 23, 3 (2013), 207–238. <https://doi.org/10.1080/08993408.2013.807962> arXiv:<https://doi.org/10.1080/08993408.2013.807962>
- [13] Juha Sorva and Teemu Sirkia. 2010. UUhistle: A Software Tool for Visual Program Simulation. In *Proceedings of the 10th Koli Calling International Conference on Computing Education Research* (Koli, Finland) (Koli Calling '10). Association for Computing Machinery, New York, NY, USA, 49–54. <https://doi.org/10.1145/1930464.1930471>
- [14] Andrew S. Tanenbaum. 2007. *Modern Operating Systems* (3rd ed.). Prentice Hall Press, USA.